

CC3092 – Deep Learning y Sistemas Inteligentes

Laboratorio #5: Agentes en el Arcade Learning Environment (ALE): Space Invaders

Angel Esteban Esquit Hernández

Repositorio: <https://github.com/AngelEsquit/Lab5-DL>

1 El Arcade Learning Environment (ALE)

Qué es ALE y relación con Stella y Atari 2600. ALE (Bellemare et al., 2013) es un framework que expone más de 50 juegos originales de **Atari 2600** como entornos estandarizados de RL, resolviendo la falta de un banco común y desafiante de tareas para comparar agentes. Se construye sobre **Stella**, un emulador de código abierto que reproduce el hardware original (CPU 6507, chip de video TIA, 128 bytes de RAM); ALE instrumenta ese emulador inyectando la acción del agente como entrada de joystick y exponiendo pantalla/RAM como observación y la variación del puntaje interno como recompensa. Los juegos son los **ROMs originales** sin modificar.

Variantes en Gymnasium. ALE/SpaceInvaders-v5 retorna el frame RGB como observación; con `obs_type="ram"` retorna los 128 bytes de RAM de la consola en su lugar. `frameskip` controla cuántos frames de emulación se repiten por cada `step()` del agente. `repeat_action_probability` (*sticky actions*, Machado et al., 2018) es la probabilidad de que el emulador ignore la acción elegida y repita la anterior, introduciendo estocasticidad para evitar que los agentes memoricen secuencias fijas explotando el determinismo de Atari. `full_action_space` expone las 18 acciones del joystick completo en vez de solo las que el juego usa.

Frame skipping. El emulador corre a 60 fps; permitir que el agente decida en cada frame sería costoso y poco informativo (frames consecutivos casi no cambian). Con frame skipping el agente elige una acción cada $k \in \{2, 3, 4\}$ frames, que se repite internamente sumando la recompensa acumulada. Esto acelera la simulación y reduce el horizonte efectivo de decisiones del episodio, facilitando el aprendizaje.

Space Invaders. El jugador mueve un cañón láser horizontalmente y dispara para destruir oleadas descendentes de invasores, evitando sus disparos y protegiéndose tras búnkeres destructibles, hasta perder sus 3 vidas o ser alcanzado por los invasores. ALE traduce el puntaje del juego directamente a **reward**: en cada `step()`, la recompensa es la diferencia de puntaje interno respecto al frame anterior; `terminated=True` cuando se pierden todas las vidas.

2 Espacios de observación y acción en entornos Atari

Observación por defecto vs. CartPole-v1. ALE/SpaceInvaders-v5 retorna un frame RGB de $210 \times 160 \times 3$ (`Box(0, 255, ..., uint8)`), mientras que **CartPole-v1** retorna un vector continuo de 4 valores (posición/velocidad del carro, ángulo/velocidad angular del poste). Trabajar con imágenes implica un espacio de estados de dimensionalidad mucho mayor (100,800 valores vs. 4): no es viable una política tabular ni una red totalmente conectada simple, se requieren aproximadores con capacidad de extraer estructura espacial (CNNs); además el agente debe aprender por sí mismo qué características visuales importan, en vez de recibirlas ya resumidas.

Observación en RAM. `obs_type="ram"` expone los 128 bytes de memoria de la consola como observación (`Box(0, 255, (128, ), uint8)`). Es preferible cuando se busca reducir drásticamente el costo computacional (permite redes pequeñas en vez de CNNs), acelerar la experimentación en hardware limitado, o cuando el interés es interpretabilidad/ingeniería de características sobre variables de estado explícitas en vez de aprendizaje de representaciones visuales.

Espacio de acción. `Discrete(6)`: `NOOP` (no hacer nada), `FIRE` (disparar sin moverse), `RIGHT/LEFT` (mover sin disparar), `RIGHTFIRE/LEFTFIRE` (mover y disparar simultáneamente).

AtariPreprocessing y FrameStackObservation. **AtariPreprocessing** aplica el preprocesamiento estándar desde DQN (Mnih et al., 2015): escala de grises, redimensionamiento a 84×84 , frame skipping (con `max` sobre los últimos frames para evitar parpadeo de sprites) y recorte opcional de recompensa a $\{-1, 0, 1\}$. **FrameStackObservation** apila las últimas k observaciones (típicamente 4) porque un solo frame no contiene información de movimiento; apilar frames permite inferir dirección/velocidad de naves y disparos sin arquitecturas recurrentes.

3 Módulo de funciones desarrollado (ale_utils.py)

Se implementó un módulo reutilizable con cinco funciones. `crear_entorno(nombre_entorno, video_folder=None, episode_trigger=None, **kwargs)` crea el entorno con `gym.make` y, si se da `video_folder`, lo envuelve con `gymnasium.wrappers.RecordVideo` (forzando `render_mode="rgb_array"`); funciona igual para ALE/SpaceInvaders-v5 que para cualquier otro entorno (verificado con CartPole-v1). `agente_aleatorio(observation, env)` retorna `env.action_space.sample()` como *baseline* sin entrenamiento. `agente_regla_simple(observation, env)` dispara continuamente (RIGHTFIRE/LEFTFIRE/FIRE) usando `get_action_meanings()` para localizar esas acciones. `ejecutar_episodio(env, funcion_agente, max_steps=10000, seed=None)` corre un episodio completo hasta `terminated`, `truncated` o `max_steps`, retornando pasos y recompensa total. `generar_video_agente(nombre_entorno, funcion_agente, video_folder, name_prefix, n_episodios=1, ...)` combina las anteriores: crea el entorno con grabación, ejecuta *n* episodios, cierra el entorno con `env.close()` (indispensable para que `RecordVideo` escriba el .mp4) y retorna rutas de video y métricas por episodio.

4 Resultados

Se generaron videos de 1 episodio completo para tres configuraciones, usando ale-py 0.12.1 y gymnasium 1.3.0:

Entorno	Agente	Pasos	Recompensa total
ALE/SpaceInvaders-v5	Aleatorio	660	210.0
ALE/SpaceInvaders-v5	Regla simple (disparo continuo)	538	270.0
CartPole-v1	Aleatorio	31	31.0

Los tres videos .mp4 quedan en la carpeta `videos/` del repositorio.

5 Discusión

El agente aleatorio logra recompensa no trivial en Space Invaders simplemente porque, de sus 6 acciones posibles, la mitad incluye disparo, por lo que dispara con relativa frecuencia y ocasionalmente acierta sin ninguna estrategia. El agente de regla simple, al disparar en *todos* los pasos en vez de repartir su presupuesto de acciones entre moverse y disparar, obtuvo mayor recompensa (270 vs. 210) en un episodio más corto (538 vs. 660 pasos, es decir, murió antes pero fue más eficiente eliminando invasores por paso). Esto confirma que una pequeña regla de dominio ("siempre disparar") supera a la acción puramente aleatoria sin necesidad de aprendizaje.

La infraestructura desarrollada (`crear_entorno`, `ejecutar_episodio`, `generar_video_agente`) es agnóstica a la política usada: basta reemplazar `funcion_agente` por un agente entrenado (p. ej. un DQN sobre observaciones preprocesadas con `AtariPreprocessing + FrameStackObservation`) para reutilizar el mismo código en el proyecto de entrenamiento de agentes.

Referencias. Bellemare, M. G., Naddaf, Y., Veness, J., & Bowling, M. (2013). The Arcade Learning Environment. *JAIR*, 47, 253–279. — Machado, M. C. et al. (2018). Revisiting the Arcade Learning Environment. *JAIR*, 61, 523–562. — Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. — Gymnasium Documentation, Farama Foundation, <https://gymnasium.farama.org/>. — ALE-py Documentation, Farama Foundation, <https://ale.farama.org/>.